Reposición asistida de repuestos mediante aprendizaje automático y coordinación multiagente en la nube

Equipo del proyecto RODA · Ingeniería de Sistemas

Informe técnico · 23 de octubre de 2026 · America/Lima

Resumen

La reposición de repuestos exige conciliar ventas, existencias, plazos de entrega, costos y restricciones de compra. Este trabajo diseñó y evaluó un prototipo académico de tienda web que integra pronóstico de demanda y coordinación multiagente para preparar decisiones administrativas verificables. Se implementó una aplicación React y FastAPI con persistencia PostgreSQL, cinco modelos de regresión comparados con referencias simples y ocho roles especializados que consultan servicios Python. La evaluación combinó una serie pública de ventas mensuales de repuestos, casos aislados de agentes y controles de permisos, errores y trazabilidad. En la comparación histórica mensual disponible, el promedio de doce meses obtuvo un error absoluto medio de 0,541 unidades por serie y mes; las versiones inicial y actualizada obtuvieron 0,575 y 0,571. En una demostración aislada, los cinco casos de control resultaron satisfactorios y la propuesta de compra permaneció pendiente de aprobación humana. Los resultados prueban la integración y sus controles bajo las condiciones ensayadas, pero no acreditan precisión de demanda para los productos actuales de RODA ni una ventaja estadística del equipo multiagente frente a reglas. La arquitectura conserva evidencia para evaluaciones posteriores con ventas observadas.

Palabras clave: pronóstico de demanda; repuestos automotrices; sistemas multiagente; inventario; evaluación temporal

1 Introducción

1.1 Problema y contexto

En una tienda de repuestos, la decisión de reponer no depende únicamente de que un producto tenga pocas unidades. Es necesario distinguir stock físico de reservas, considerar compras pendientes, comprobar si una oferta sigue vigente y determinar cuánto permite el presupuesto. Una decisión apresurada puede duplicar un pedido a proveedor o dejar un faltante sin advertir. El presente estudio aborda esa tarea administrativa en un prototipo académico peruano denominado RODA, sin atribuir sus registros de ejemplo a una empresa real.

El diagnóstico provino de revisar los flujos implementados del propio prototipo: catálogo, pedidos, kardex, proveedores y panel de administración. Se identificó la necesidad de convertir registros dispersos en una propuesta examinable por una persona responsable. La pregunta técnica fue si un pronóstico, cuando su evidencia fuese válida, y un equipo de roles con herramientas acotadas podían preparar una recomendación reproducible sin asumir la autoridad para ejecutar una compra. No se realizaron encuestas ni se midieron procesos de una empresa externa.

La salida esperada especifica producto, cantidad, proveedor, importe, plazo, advertencias y fundamento. Registra herramientas consultadas y observaciones resueltas. Si falta un pronóstico válido, aplica la política de inventario; si cambian stock u oferta antes de aprobar, exige un análisis nuevo. Estos criterios hacen evaluable el prototipo.

1.2 Antecedentes y fundamento

El archivo Monash reúne series de distintas frecuencias y publica referencias de pronóstico para facilitar comparaciones entre métodos; su aporte metodológico es separar el comportamiento de una serie de la fama de un algoritmo (Godahewa et al., 2021). El conjunto Car Parts asociado contiene 2674 series mensuales intermitentes de ventas de repuestos entre enero de 1998 y marzo de 2002 (Godahewa et al., 2020). Es una fuente externa útil para estudiar la evaluación temporal, aunque sus identificadores anónimos no corresponden a los SKU de RODA ni permiten inferir precios, stock o demanda peruana actual.

En pronóstico, una referencia ingenua o promedio histórico sirve para comprobar si la complejidad adicional aporta valor; el error se calcula sobre observaciones posteriores al entrenamiento (Hyndman y Athanasopoulos, 2021). Además, imputar o transformar datos antes de separar entrenamiento y prueba puede filtrar información futura y producir resultados optimistas (Scikit-learn developers, s. f.-a). Por ello, el trabajo conserva periodos distintos para entrenamiento, actualización y comparación, y no reemplaza valores de demanda desconocidos por ventas inventadas.

En sistemas multiagente, AutoGen presenta agentes conversacionales configurables que combinan herramientas, modelos y participación humana (Wu et al., 2023). Se tomó como antecedente conceptual de cooperación, no como biblioteca instalada. RODA usa una implementación propia: un orquestador selecciona especialistas, cada especialista interpreta datos de un servicio Python y el auditor revisa el

expediente. Se trata de un flujo secuencial y jerárquico con decisiones acotadas, no de ocho modelos distintos ni de una afirmación general de superioridad del paradigma multiagente.

La comparación de antecedentes se resume en la Tabla 7 del anexo B: se distinguen propósito, datos, técnica, evaluación, resultado y limitación. El vacío atendido es unir evidencia predictiva, restricciones transaccionales y revisión administrativa en un expediente reproducible, sin atribuir al prototipo resultados de los trabajos citados.

Los roles utilizan un modelo de lenguaje de gran tamaño (LLM) y se orientan a un objetivo acotado: consultan herramientas autorizadas, interpretan observaciones y eligen una decisión tipada. Su autonomía reside en delegar, seleccionar alternativas y pedir correcciones; el servidor conserva la autoridad de validar y escribir. El aprendizaje automático (ML) calcula cantidades mediante un contrato separado.

Objetivos verificables

1. Diseñar e implementar una tienda web académica que registre ventas, inventario y propuestas administrativas trazables, con separación entre interfaz, servicios y persistencia.

2. Comparar cinco modelos predictivos con referencias históricas mediante una división temporal y establecer cuándo un resultado puede o no respaldar una reposición.

3. Evaluar una coordinación multiagente de ocho roles mediante casos normales, fallos controlados, permisos y comparación exploratoria con reglas y un solo agente.

2 Materiales y métodos

2.1 Diseño y alcance del estudio

Se realizó un desarrollo tecnológico aplicado con evaluación exploratoria. La unidad de análisis predictivo fue una serie de ventas por identificador y periodo; la unidad de análisis multiagente fue una ejecución administrativa con entradas, pasos, controles y salida. Para la comparación mensual se utilizaron datos públicos observados; para las pruebas de flujos de compra se usó una base temporal con productos y ofertas académicas identificadas. Ambos ámbitos permanecen separados. La evaluación no pretendió estimar el beneficio económico de una tienda en operación.

Se establecieron criterios previos: registrar la procedencia de datos, reservar periodos futuros, comparar MAE con una referencia simple, impedir que una propuesta exceda el presupuesto y mantener aprobación humana antes de crear órdenes de compra. Los casos de error incluyeron una oferta inválida, evidencia de stock cambiada, respuesta JSON incorrecta y fallo temporal del proveedor LLM. El éxito de una prueba se definió por controles observables, no por una explicación textual persuasiva.

2.2 Arquitectura y controles cloud

La Figura 1 muestra el recorrido entre interfaz, servicios, persistencia y componentes inteligentes. El modelo predictivo y el LLM tienen contratos distintos. El primero calcula cantidades; el segundo interpreta consultas y devuelve decisiones estructuradas que el servidor valida.

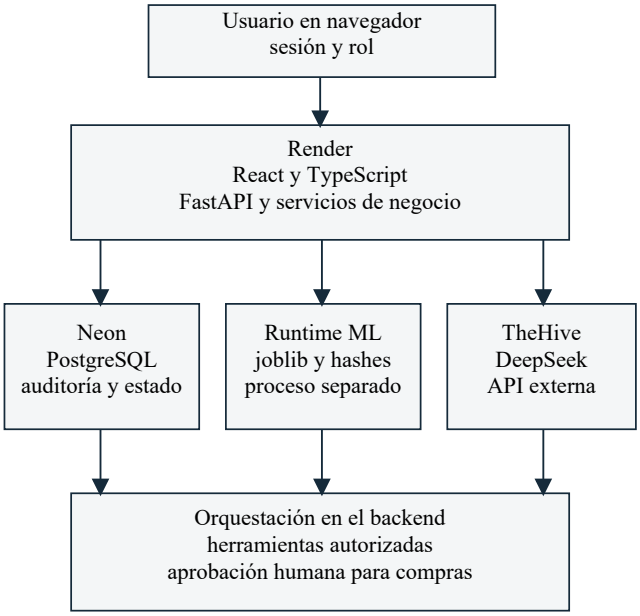


Figura 1. Arquitectura del prototipo desplegado. Fuente: elaboración propia a partir del código y render.yaml.

La Figura 1 representa cinco límites: navegador, API, servicios de negocio, persistencia y proveedores externos. El frontend React con TypeScript presenta tienda y área administrativa; FastAPI autentica las solicitudes y deriva el trabajo a módulos Python. PostgreSQL en Neon conserva productos, pedidos, movimientos, ofertas, sesiones y ejecuciones de agentes. El servicio web y la compilación del frontend están definidos en render.yaml para Render. El entorno gratuito es apropiado para demostraciones, pero puede suspenderse tras inactividad y su sistema de archivos no ofrece persistencia permanente (Render, s. f.).

Los servicios Python calculan cantidades y controles autoritativos. El proveedor de lenguaje devuelve decisiones estructuradas en JSON, pero no ejecuta SQL arbitrario ni crea una orden por sí mismo. El orquestador selecciona un equipo según tipo de tarea; cada especialista recibe mensajes breves y la salida de herramientas permitidas para su rol. El auditor recibe un expediente que reúne plan, alternativas, calidad, finanzas, advertencias y correcciones. agent_runs conserva estado, llamadas, tokens y

checkpoint; agent_steps conserva cada consulta, mensaje y revisión.

Los agentes de reposición tienen una dependencia causal: Calidad comprueba inconsistencias, Inventario calcula necesidad, Compras compara alternativas, Finanzas revisa límite y margen, y Auditor verifica el expediente. Demanda ML se incorpora únicamente si existe pronóstico operativo vigente; Pedidos y envíos se incorpora si hay incidencias. Otros tipos de tarea, como catálogo o informe financiero, usan equipos más pequeños. El administrador puede iniciar, cancelar o retomar un análisis fallido; una revisión diaria opcional solo encola un análisis, sin emitir compras.

En la configuración protegida, el acceso a las rutas de agentes exige sesión y rol administrador. La cookie es HttpOnly y SameSite=Lax, con atributo Secure bajo HTTPS. Las escrituras rechazan orígenes ajenos y las variables de conexión o proveedor se leen del entorno. La propuesta validada queda en estado de revisión; aprobarla requiere un acto explícito del administrador y una transacción que vuelve a comprobar vigencia de ofertas e inventario. La interfaz no debe confundir este paso con pago, recepción o entrega automática.

La persistencia externa se apoya en la separación de cómputo y almacenamiento de Neon (Neon, s. f.). La ejecución de evaluación registró Python 3.12.10, scikit-learn 1.7.2, numpy 2.2.6, pandas 2.2.3 y joblib 1.5.1. Los contratos de datos y las dependencias fijadas permiten repetir los cálculos con esas versiones.

requirements.txt fija FastAPI 0.135.1 y Uvicorn 0.41.0. El archivo de bloqueo del frontend registra React 19.3.0, TypeScript 5.9.3 y Vite 6.4.4. La configuración utiliza DATABASE_URL, LLM_PROVIDER, THEHIVE_API_KEY, THEHIVE_MODEL, ASSISTANT_ENABLED y SEED_PASSWORD; DEMO_ACCESS_ENABLED controla el acceso temporal de exposición. Solo se publican sus nombres.

2.3 Datos y modelo predictivo

La fuente externa principal fue Car Parts, publicada en Zenodo por Godahewa y colaboradores. Registra 2674 series mensuales anónimas y contiene ausencias; el archivo preparado documenta 136374 filas y 6122 valores faltantes. El historial inicial, los datos incorporados después y las ventas posteriores se guardaron por separado, con sumas de verificación. Los signos de ausencia permanecieron como datos desconocidos. Se excluyeron objetivos no observados de la comparación y se imputaron entradas solo dentro del entrenamiento. No se asignaron series externas a productos del catálogo ni se derivaron precios o existencias inexistentes.

La variable objetivo fue la cantidad vendida en el siguiente mes para cada serie. Las entradas procedieron únicamente de ventas anteriores al corte: rezagos de uno, dos, tres, seis y doce meses; promedio, dispersión, venta reciente, proporción de ceros, número de faltantes y posición estacional del mes. El modelo no recibió el valor posterior que debía predecir. La prueba final empleó el mismo conjunto de 15054 observaciones, correspondientes a 2509 series y seis meses entre octubre de 2001 y marzo de 2002, para comparar versiones y promedio histórico.

Se evaluaron cinco familias: Ridge como relación lineal regularizada, Poisson para conteos, Random Forest y Extra Trees para interacciones no lineales, e Hist. Gradient Boosting como ensamble secuencial. Los hiperparámetros se fijaron o eligieron usando historial anterior a la prueba. V1 se entrenó con el primer tramo; V2 añadió meses conocidos y volvió a ajustar el modelo. La referencia fue el promedio histórico de los últimos doce meses observables. El MAE en unidades por serie y mes fue la métrica primaria; RMSE, WAPE, sesgo y R^2 ayudaron a interpretar errores sin redefinir el ganador tras mirar la prueba.

La web tiene además un contrato independiente para un pronóstico de siete días por SKU de su catálogo académico. Allí se compararon cinco modelos y referencias con cortes temporales, y el artefacto desplegado identifica Ridge como candidato académico. Esa evaluación no debe mezclarse numéricamente con Car Parts mensual: cambian frecuencia, población, objetivo y unidad. El servicio de inventario solo aplica un pronóstico operativo de la fecha anterior, con hash de modelo coincidente y predicción sin observación; de otro modo usa mínimos, máximos y compras pendientes. La demostración de ML repite un pronóstico guardado en una base temporal y no representa un entrenamiento nuevo.

V1 utilizó enero de 1998 a diciembre de 2000 y validó julio a diciembre de 2000. V2 amplió el historial hasta septiembre de 2001 y validó abril a septiembre de 2001. La prueba reservó octubre de 2001 a marzo de 2002. Cada origen mensual requiere una ventana de doce meses con al menos ocho cantidades conocidas; los objetivos desconocidos no se imputan.

La Tabla 1 define la unidad de observación, los predictores y el tratamiento de valores desconocidos. La Tabla 2 detalla la configuración que se mantuvo fija durante la comparación.

Tabla 1. Variables del experimento mensual Car Parts

Variable	Definición y tratamiento
sku y periodo	Clave de serie anónima y mes AAAA-MM; identifican filas y cortes. El SKU no entra como predictor numérico.
unidades	Conteo mensual observado. Cero significa ausencia de ventas; vacío significa desconocido. Objetivo: unidades del mes siguiente.

Variable	Definición y tratamiento
lag1, lag2, lag3, lag6, lag12	Cantidades observadas a 1, 2, 3, 6 y 12 meses antes del mes previsto. Nulos de entrada se imputan con la media dentro del pipeline.
mean, std, recent	Media y desviación de la ventana de 12 meses; media de los últimos 3. Unidad: unidades por serie y mes.
zero_share, missing	Proporción de ceros entre datos observados y número de faltantes en la ventana. Sin confundir desconocidos con cero.
month_sin y month_cos	Codificación cíclica del mes previsto. Valores adimensionales calculados con el calendario.

Los rezagos representan persistencia y recurrencia; las estadísticas de ventana resumen nivel y variación; la proporción de ceros describe intermitencia; los faltantes distinguen calidad de observación de ausencia de ventas; seno y coseno representan el calendario sin un salto artificial entre diciembre y enero. Estas señales describen ventas observadas, no demanda perdida por quiebre de stock.

Tabla 2. Configuración común de los cinco candidatos

Algoritmo	Configuración fijada
Ridge	Media, indicador de faltantes, StandardScaler; alpha=10.
Poisson	Media, indicador, escalado; alpha=0.1; max_iter=500.
Random Forest	60 árboles, profundidad 10, mínimo 5 por hoja; n_jobs=2; semilla 42.
Extra Trees	60 árboles, profundidad 10, mínimo 5 por hoja; n_jobs=2; semilla 42.
Hist Gradient Boosting	150 iteraciones, 31 hojas, regularización L2=1; semilla 42; imputación con media.

Los doce predictores y los hiperparámetros corresponden al laboratorio mensual, no al motor diario de referencia ni al conjunto extendido experimental. Todos los candidatos incluyen imputación con media e indicadores de faltantes. La selección usa el menor MAE de validación; el ganador se reajusta con los objetivos conocidos hasta el corte. La Tabla 11 del anexo C presenta los resultados individuales de los cinco candidatos y la referencia en cada ventana. Las estimaciones negativas se recortan a cero. El artefacto conserva variables, frecuencia, fecha, series admitidas y hash SHA-256.

MAE, RMSE y R^2 describen errores de regresión con propiedades distintas; la métrica principal permanece fijada antes de la prueba (Scikit-learn developers, s. f.-b). No se cambia el criterio después de observar qué alternativa gana.

2.4 Agentes y supervisión humana

El sistema define ocho roles no redundantes: Orquestador, Calidad de datos, Demanda ML, Inventario, Compras y proveedores, Finanzas, Pedidos y envíos, y Auditor. Cada rol dispone de una lista blanca de herramientas de lectura, una evidencia de entrada y una salida tipada que expresa continuar, revisar o bloquear. Todos pueden compartir el mismo modelo de lenguaje configurado; la especialización reside en su responsabilidad, contexto y herramienta autorizada. Los cálculos de stock, costo y límites permanecen en los servicios Python.

Orquestador consulta el contexto y fija estrategia de costo o rapidez. Calidad consulta errores y origen; Demanda interpreta vigencia de pronósticos sin extrapolar. Inventario produce candidatos y cantidades netas de reservas y compras en tránsito. Compras inspecciona ofertas activas con costo, lote mínimo, plazo y procedencia; si elige otra oferta, el servidor valida que el identificador figure entre las alternativas. Finanzas compara el plan con el presupuesto y detecta costos mayores o iguales al precio de venta. Pedidos identifica atrasos e incidencias sin cambiar estados. Auditor contrasta restricciones y observaciones antes de presentar la propuesta.

Una oferta etiquetada referencia_catalogo es un costo académico derivado del catálogo, no una cotización recibida. Si existe una oferta manual viable, la referencia no sustituye esa oferta. El plan de compras prioriza productos secuencialmente; por tanto, no asegura la combinación global óptima. Cuando el presupuesto solo cubre una parte, las unidades restantes quedan visibles como advertencia. El flujo puede solicitar una corrección limitada a Compras y una nueva lectura de Finanzas; una inconsistencia real o una alternativa dominada sin justificación bloquea el cierre.

El control técnico limita cada análisis a treinta llamadas LLM, seis minutos y veinte productos; cada intervención dispone de hasta tres turnos. Las decisiones JSON pasan por validación de esquema. Una selección inválida se devuelve para corrección acotada, mientras que un cambio real de inventario detiene el análisis. Se registran llamadas, errores y pasos sin atribuir a los textos del proveedor autoridad sobre la base de datos. La propuesta de reposición únicamente puede transformarse en órdenes mediante aprobación humana, con validación transaccional de datos vigentes e idempotencia.

La Tabla 8 del anexo B especifica, para cada uno de los ocho roles, objetivo, entrada, herramienta autorizada, salida, límite y criterio de aceptación. Tener ocho roles disponibles no obliga a invocarlos en todas las tareas; un rol sin evidencia suficiente no debe producir una conclusión inventada.

2.5 Protocolo de evaluación

La evaluación predictiva separó cronológicamente entrenamiento, actualización y comparación posterior. Se calculó MAE sobre los mismos pares serie-mes para V1, V2 y referencia, sin seleccionar una versión con filas distintas. Para analizar la repetibilidad se conservaron notebooks, archivos de entrada, hashes, parámetros, modelos serializados y salidas de evaluación. Esta estructura permite ejecutar la carga de datos adicionales sin alterar el historial inicial ni prometer que la siguiente versión mejorará automáticamente.

En agentes se prepararon cinco casos académicos aislados: ajuste por presupuesto, contraste costo-plazo de proveedores, preguntas difíciles al asistente, reserva inconsistente y efecto de un pronóstico vigente sobre la cantidad. La ejecución usa SQLite temporal y guarda un resultado resumido en la aplicación. Tres controles adicionales verifican presupuesto, compra en tránsito y reserva sin pedido con reglas Python, sin llamadas LLM. Para contraste exploratorio se repitieron tres escenarios en modalidades reglas, un agente y equipo completo, registrando estado, llamadas, tokens y segundos.

Se añadieron pruebas de permisos y fallos: empleado sin aprobación de compras, cambio de stock entre propuesta y aprobación, oferta vencida, rollback ante fallo del segundo proveedor, JSON inválido, selección de oferta ajena y caída temporal de API. Una prueba confirma que una petición maliciosa al asistente no crea venta ni cambia stock. La aceptación exige el resultado previsto y ausencia de efectos no autorizados. El tiempo de ejecución se informa como latencia observada de una configuración y no como garantía de desempeño en otras redes o proveedores.

El estudio nuevo fijó dos repeticiones por escenario y modalidad, límite de 360 s por análisis y controles de presupuesto, vigencia y ausencia de compras automáticas. Para las ocho preguntas del asistente se fijó un límite de 30 s por caso y se verificó información de la base, respuesta del proveedor y ausencia de costos privados.

Las pruebas automáticas de permisos, integridad y recuperación utilizan SQLite temporal y respuestas controladas; se separan de las llamadas reales del estudio del proveedor. El criterio funcional exige satisfacer todos los controles del caso. El tiempo se mide de inicio a término, incluyendo proveedor. Los percentiles de una muestra pequeña no se interpretan como garantía de servicio.

La Tabla 9 del anexo C presenta entradas, expectativas, aceptación, resultados y archivos de evidencia para casos normales, límites, error, privacidad y desempeño. Todas sus filas identifican fecha y versión. El protocolo conserva también los fallos; un resultado incompleto no se convierte en un caso satisfactorio.

El 23 de octubre se repitieron 41 pruebas de agentes en 13.137 s y 11 del asistente en 1.555 s; todas finalizaron correctamente. Cubrieron rechazo por rol, aprobación atómica e idempotente, stock cambiado, JSON inválido, recuperación acotada y ausencia de costos privados en respuestas al cliente. Estos tiempos corresponden a pruebas locales con proveedor aislado, no a latencia del LLM real. Los archivos pruebas-agentes.txt y pruebas-asistente.txt conservan nombres, resultado y duración.

3 Resultados y discusión técnica

3.1 Evaluación temporal del aprendizaje automático

La Tabla 3 compara las mismas 15,054 observaciones posteriores de 2,509 series, entre 2001-10 y 2002-03. El avance temporal incorpora únicamente ventas ya observadas antes de cada mes. No constituye una predicción fija de seis meses realizada de una sola vez.

Tabla 3. Prueba temporal común y referencia histórica

Método	MAE	RMSE	R^2
V1	0.5747	1.0340	0.1023
V2	0.5710	1.0331	0.1038
Promedio	0.5411	1.0460	0.0812

Fuente: comparison.json de la ejecución local del 23/10/2026; n=15,054 pares serie-mes. MAE y RMSE: unidades por serie y mes; R^2: adimensional. V1 y V2 usan exactamente los mismos objetivos observados.

V1 eligió Hist. Gradient Boosting y V2 eligió Extra Trees usando sus validaciones respectivas. El menor MAE de prueba corresponde a Promedio. Un cálculo válido o un modelo ganador entre cinco candidatos no equivalen a superar una referencia simple. MAE se expresa en unidades por serie y mes; WAPE no es un porcentaje de exactitud.

El 79.53% de los objetivos de prueba son cero. Predecir siempre cero obtiene MAE 0.3867, inferior a los métodos de la Tabla 3, pero omite todas las ventas positivas. Se conserva como diagnóstico de dispersión, no como política de reposición. El promedio es la referencia operativa predefinida; que los modelos tampoco superen esta referencia exige mantenerlos como exploratorios.

En los 3,081 objetivos con ventas positivas, V2 registró MAE 1.2909 y sesgo -1.2718 unidades: subestimó esas ventas. El análisis por segmento impide interpretar un error global bajo, dominado por ceros, como capacidad suficiente de abastecimiento. Los segmentos y su tamaño se conservan en comparison.json.

La Figura 2 representa el MAE de las dos versiones y de la referencia sobre la misma prueba temporal.

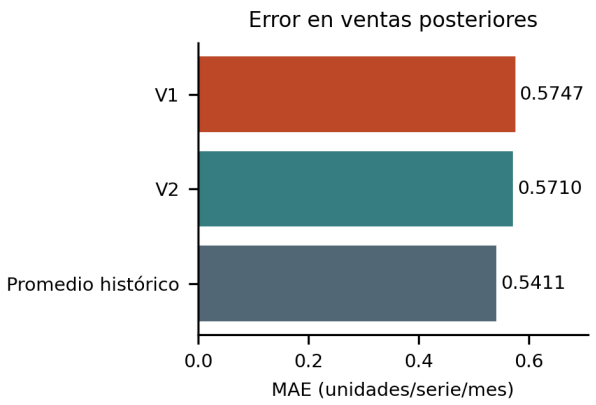


Figura 2. Error de predicción y comparación de versiones. Menor MAE es mejor. Fuente: evaluación temporal reproducible.

La reducción relativa de MAE de V2 frente a V1 fue 0.646%, por debajo del umbral de promoción del 1%. Además, ninguna versión superó el promedio. El promedio alcanzó menor MAE, aunque V2 tuvo menor RMSE y mayor R^2: esta diferencia refleja que las métricas penalizan de forma distinta los errores grandes. La selección conserva MAE como criterio previo.

La ejecución completa del 23 de octubre duró 469.53 s localmente: 11 celdas sin error, diez ajustes para seleccionar cinco candidatos por versión y dos reajustes finales. La lectura del Excel verificó 24,066 filas contra el CSV original, incluidos los faltantes. MAE, RMSE y R^2 reprodujeron exactamente los resultados anteriores. Se comprobó la preservación SHA-256 de 17 archivos originales. El cuaderno ejecutado conserva estas salidas; no se atribuye su ejecución a Colab ni a Render.

3.2 Comparación de modalidades y coordinación

La Tabla 4 presenta cada ejecución del equipo. Debajo se resumen las mismas pruebas con reglas y con un solo agente.

Tabla 4. Casos del equipo multiagente y resultado observado

Caso y rep.	Estado	Tiempo s	Solicitudes
presupuesto 1	Cumple	58.58	8
presupuesto 2	Cumple	75.19	9
ofertas 1	Cumple	50.09	7
ofertas 2	Cumple	58.59	8
vencidas 1	Cumple	32.66	5
vencidas 2	Cumple	33.83	5

reglas: 6/6 cumplen; tiempo medio 0.06 s; 0 solicitudes al proveedor.

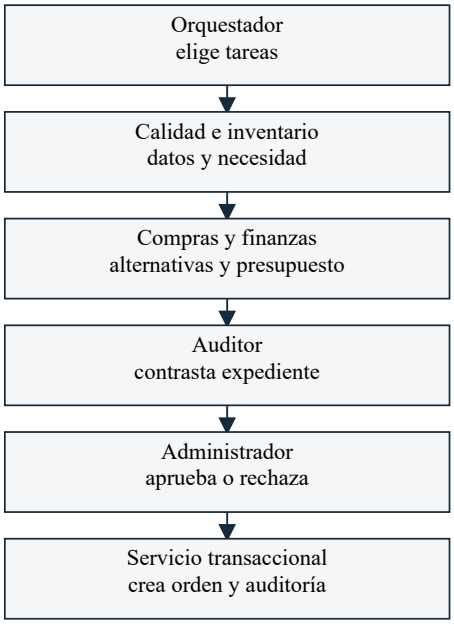
un_agente: 4/6 cumplen; tiempo medio 23.19 s; 12 solicitudes al proveedor.

multiagente: 6/6 cumplen; tiempo medio 51.49 s; 42 solicitudes al proveedor.

Resultado esperado: presupuesto, propuesta dentro del límite; ofertas, selección válida; vencidas, bloqueo sin compras. Cada fila se identifica por caso y repetición en agents/entrega-2026-10-23.json, dentro de los anexos. Se conservan entrada, criterios, resultado, proveedor, fecha, versión y evidencia. Los tokens desconocidos de intentos fallidos no se contabilizan como gratuitos.

La comparación nueva contiene 18 ejecuciones. Los casos prueban presupuesto, alternativas de proveedor y vigencia de ofertas. Las tres modalidades comparten servicios y controles Python. El denominador incluye fallos de formato y errores del proveedor, sin descartarlos. Los tiempos y tokens completos se incluyen en el archivo de evidencia.

La Figura 3 explica qué se coordina: una necesidad de inventario, alternativas de compra, restricciones y revisión. Los agentes utilizan el mismo proveedor LLM con contextos y herramientas diferentes; no son ocho modelos entrenados independientemente. Las decisiones observables y las herramientas se registran, sin publicar razonamiento interno del proveedor.



Demanda y pedidos participan si la tarea los requiere.

Figura 3. Coordinación para preparar una compra supervisada. Fuente: orquestador y servicios del prototipo.

En la demostración aislada de Render del 22 de octubre, los cinco casos de control se completaron. El caso de presupuesto propuso S/ 40.00 dentro de un límite de S/ 50.00 y conservó la aprobación humana. La exportación registra ocho llamadas del equipo. Esta prueba usa datos académicos temporales y no representa una compra comercial ni demuestra ahorro operativo real.

El siguiente fragmento adapta las llamadas del comparador a los servicios compartidos; la creación de una compra permanece en otra operación autenticada.

```
stock = data.candidates(5)
state['plan'] = data.purchase_plan(
    stock, 'costo', budget, True)
state['audit'] = data.audit_result(
    run, state)
```

3.3 Evaluación del asistente de la tienda

La Tabla 5 presenta los ocho casos del asistente, su cumplimiento y la latencia observada con el proveedor real.

Tabla 5. Consultas del asistente y controles observados

Caso	Resultado	Tiempos
precio	Cumple	4.875
incompatible	Cumple	3.438
ambiguo	Cumple	1.672
contexto	Cumple	2.75
seguimiento	Cumple	2.844
inexistente	Cumple	3.047
manipulacion	Cumple	3.562
tienda	Cumple	3.219

Se registraron 8 casos, de los cuales 8 cumplieron los criterios definidos. La evaluación conserva solicitudes, verificaciones y proveedor efectivo sin claves. El catálogo es un conjunto de prueba aislado; el servicio no inventa ventas ni modifica el stock para responder consultas.

3.4 Evidencia del despliegue

La Tabla 6 separa consultas y pruebas completadas en Render de los pasos que no finalizaron en la instancia gratuita.

Tabla 6. Recorrido comprobado en el sistema desplegado

Prueba	Estado y alcance
Arranque tras suspensión	23/10: Render mostró su página de arranque; después abrió el laboratorio con sesión administradora.
Persistencia del laboratorio	23/10: se restauraron 120,330 observaciones, V1 y el pronóstico guardado del 22/10.
Excel y predicción V1	22/10: importación de 24,066 filas; pronóstico guardado con 2,509 resultados válidos y CSV descargado.
Intento de entrenamiento V2	23/10 a las 11:34:51: se inició en Render. Terminó como Revisar: el servidor se reinició. No se obtuvo V2 en esta prueba cloud.
Evaluación posterior cloud	No ejecutada: depende de que termine V2. El ciclo completo se entrenó y evaluó localmente el 23/10, con notebook ejecutado adjunto.
Agentes y asistente	Exportación Render del 22/10: 5/5 casos aislados. El 23/10 se consultó la propuesta guardada: seis agentes, 23 unidades, S/ 1,617.21 de S/ 2,000, por aprobar y sin pronóstico operativo.
Asistente web 23/10	Consulta RP-010-01: respuesta TheHive/DeepSeek, S/ 160.00, 14 disponibles y compatible con Toyota Yaris 2018 según catálogo. Sin errores de consola capturados.

La disponibilidad depende del plan y de la API externa. Render Free puede suspender la instancia por inactividad y su disco es efímero (Render, s. f.). El prototipo persiste el laboratorio y sus modelos en Neon y los restaura al iniciar. Esta recuperación no equivale a garantizar un planificador activo durante toda la jornada.

La prueba de V2 del 23 de octubre no llegó a una versión guardada en Render: el historial informó un reinicio del servidor. Por ello, no se ejecutó allí la comparación posterior. No se infiere la causa exacta del reinicio sin registros suficientes. La evidencia reproducible local cierra el ciclo completo; para una exposición cloud, el entrenamiento grande debe ejecutarse fuera de la instancia gratuita o en un entorno cuya capacidad se verifique antes.

Para la entrega se habilitó un acceso de demostración con dos botones, administrador y cliente, sin contraseña. Se conservaron las sesiones y los controles asociados al rol, pero se permitió que cualquier visitante eligiera administrador. Es una configuración de exposición, sin protección de identidad. Se desactiva con DEMO_ACCESS_ENABLED=false, que invalida sus sesiones temporales; las contraseñas originales permanecen. La Figura 7 y render-demo.json documentan la comprobación final del acceso.

En la revisión final del acceso se completaron 12/13 comprobaciones satisfactorias. Las consultas de pronósticos, evaluación, laboratorio y expedientes de agentes respondieron correctamente. La sesión cliente recibió 403 al intentar consultar el ML administrativo. Estos controles son de consulta y permisos; no crean ventas ni órdenes.

La consulta comercial final tardó 14.157 s y respondió mediante catálogo, no mediante el LLM. El precio y la disponibilidad se conservaron, pero ese resultado no satisface el criterio de respuesta del proveedor externo. La configuración de una API no garantiza que atienda una solicitud. Se conserva este fallo junto a las ocho consultas anteriores correctas, sin reemplazarlo por ellas.

En una comprobación posterior, fechada el 23/10/2026 a las 13:58, la misma consulta obtuvo HTTP 200 mediante TheHive en 10.703 s, con precio S/ 160.00 y disponibilidad de 14 unidades. Se endureció la validación del adaptador para rechazar respuestas incompletas o ambiguas y registrar metadatos seguros. La causa del primer JSON inválido no se confirmó; una respuesta posterior correcta no prueba que el cambio haya causado la recuperación ni garantiza disponibilidad continua. render-provider-recovery.json conserva este resultado por separado.

3.5 Interpretación y limitaciones

La separación entre decisiones de lenguaje y cálculos autoritativos permitió inspeccionar por qué una propuesta pasó, se corrigió o se bloqueó. Sin embargo, la validez de una cadena de agentes no se deduce de que todos hayan escrito mensajes: importan las herramientas consultadas, los valores contrastados y el estado final. Los controles de presupuesto y auditoría habrían funcionado también como reglas Python; la aportación observable de la orquestación es organizar especialidades y dejar un expediente legible, a cambio de más llamadas y latencia.

La evidencia predictiva real es histórica y mensual, mientras que la reposición web necesita estimaciones por SKU y horizonte corto. Car Parts carece de nombres de piezas, precio, existencias y país verificable para cada serie. No es correcto transferir su MAE al catálogo de RODA ni afirmar que V2 aprendió de ventas actuales. El historial de la tienda es académico y la propia pantalla distingue pronósticos operativos de demostraciones. Una evaluación futura debería recopilar cierres observados, verificar calidad y ausencia de quiebres no registrados y repetir pruebas temporales antes de autorizar cambios de modelo.

El proveedor LLM puede fallar por latencia, formato o disponibilidad. Se acotaron llamadas, turnos y reintentos, y los fallos se registran sin aprobar una compra. El plan gratuito de Render puede suspender el servicio inactivo y no conserva archivos locales tras reinicios; PostgreSQL externo mantiene datos persistentes, pero una revisión programada depende de que el servicio esté activo o vuelva a iniciarse. Por ello, una demostración académica debe comprobar el estado del despliegue antes de ejecutarse y conservar exportaciones de resultados sin secretos.

La comparación nueva del 23 de octubre amplió a dos repeticiones por caso y modalidad: reglas 6/6, agente único 4/6 y equipo 6/6. El agente único agotó reintentos de formato en los dos casos de presupuesto; el equipo recuperó tres salidas JSON inválidas y completó los casos. La muestra comparte tres escenarios fijos, por lo que la diferencia observada es exploratoria y no demuestra superioridad estadística. El uso de llamadas, latencia y tokens debe valorarse junto con la revisión especializada.

Los fallos del agente único se registraron como ProviderUnavailable después de tres errores de decodificación JSON en cada caso. No se confirmó truncamiento ni una causa única del formato recibido; el estudio conserva el fallo observable sin atribuirlo a una causa no demostrada.

4 Conclusiones

Se implementó una aplicación web que integra catálogo, inventario, pedidos y propuestas administrativas con una capa de servicios Python y persistencia relacional. La separación permitió que herramientas de agentes reutilizaran cálculos de negocio y que una propuesta no modificara stock ni creara compras sin autorización. El alcance demostrado es el de un prototipo académico desplegado, no el de una tienda comercial evaluada en producción.

La comparación histórica mensual permitió entrenar y actualizar dos versiones sobre ventas observadas externas y compararlas con una referencia simple. En 15054 observaciones posteriores, V2 alcanzó MAE 0,571 y el promedio histórico 0,541; por ello, el experimento no justifica promover V2 como mejor método para ese conjunto. El resultado demuestra un criterio de selección prudente, sin validar precisión para SKU peruanos actuales ni para un horizonte semanal.

La coordinación de ocho roles produjo propuestas auditadas y mantuvo supervisión humana. En la comparación nueva, el equipo cumplió 6/6 ejecuciones, las reglas 6/6 y el agente único 4/6; este último presentó dos fallos de formato del proveedor. Los ocho casos del asistente cumplieron sus controles. Los resultados respaldan la integración bajo los escenarios ensayados, pero el mayor tiempo y número de llamadas impiden atribuir una ventaja general al equipo.

El aporte principal es una arquitectura evaluable con procedencia de datos, controles de acceso, límites de llamadas, detección de evidencia cambiante y trazabilidad de decisiones. Para avanzar se recomienda acumular ventas observadas, comparar pronósticos por SKU en varios cortes y repetir escenarios de agentes con más instancias y métricas de calidad además de tiempo y costo. Hasta entonces, las propuestas deben conservar la revisión del administrador.

Referencias

Godahewa, R., Bergmeir, C., Webb, G., Hyndman, R., y Montero-Manso, P. (2020). Car Parts Dataset (with Missing Values) (versión 2) [Conjunto de datos]. Zenodo. https://doi.org/10.5281/zenodo.4656022

Godahewa, R., Bergmeir, C., Webb, G. I., Hyndman, R. J., y Montero-Manso, P. (2021). Monash Time Series Forecasting Archive. arXiv. https://doi.org/10.48550/arXiv.2105.06643

Hyndman, R. J., y Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3.ª ed.). OTexts. https://otexts.com/fpp3/toolbox.html

Neon. (s. f.). Architecture overview. Recuperado el 23 de octubre de 2026 de https://neon.com/docs/introduction/architecture-overview

Render. (s. f.). Deploy for Free. Recuperado el 23 de octubre de 2026 de https://render.com/docs/free

Scikit-learn developers. (s. f.-a). Common pitfalls and recommended practices. Recuperado el 23 de octubre de 2026 de https://scikit-learn.org/stable/common_pitfalls.html

Scikit-learn developers. (s. f.-b). Metrics and scoring. Recuperado el 23 de octubre de 2026 de https://scikit-learn.org/stable/modules/model_evaluation.html

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., y Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv. https://doi.org/10.48550/arXiv.2308.08155

Anexo A Acceso y reproducción

Sistema: https://roda-repuestos.onrender.com
Repositorio: https://github.com/NEYEN111/REPUESTOS
Colab: https://colab.research.google.com/github/NEYEN111/REPUESTOS/blob/main/notebooks/RODA_ML_Datos_Reales.ipynb

El paquete adjunto incluye el cuaderno ejecutado, insumos CSV y Excel, modelos joblib de la evaluación, resultados JSON y archivos de pruebas. El archivo README indica dependencias, comandos y variables requeridas. Los identificadores de ejecución y hashes se conservan en los artefactos. Las claves API permanecen fuera del informe. Durante el acceso temporal de exposición no se requiere contraseña para elegir administrador o cliente; fuera de ese modo se conserva el inicio de sesión original.

Recorrido de exposición: mostrar la fuente y los cortes; abrir V1; importar el Excel; ejecutar o consultar V2; incorporar ventas posteriores; comparar con la referencia; abrir el expediente de agentes; explicar una corrección y el control del Auditor; consultar precio o compatibilidad en el asistente. Una función que no complete la prueba en Render se identifica en la Tabla 6 y se demuestra por separado con evidencia local.

La ejecución de resultados está fijada al código a89d21d38dd88f8f38dc3dcefefa5b2045a9c0f5 y a los archivos fechados el 23/10/2026. Las mejoras posteriores de acceso de demostración no cambian esos resultados experimentales. El manifiesto del ZIP permite comprobar tamaño y SHA-256 de cada artefacto. No se incluyen variables de entorno con sus valores, claves de proveedor ni datos personales.

La última consulta desplegada verificada corresponde a la revisión 0243a2aab72018b5304a5464839f75fb2b2da3f0, fechada el 23/10/2026. Su evidencia de recuperación es independiente del código de los experimentos históricos y se conserva en render-provider-recovery.json.

El código completo de la aplicación se obtiene del repositorio GitHub. El ZIP es el paquete de informe, datos, modelos y evidencias, con los componentes necesarios para revisar el experimento; no sustituye el clon del repositorio para iniciar toda la tienda. Los comandos de reproducción se ejecutan desde ese clon, según el README.

Anexo B Antecedentes y contrato de los agentes

La Tabla 7 compara los antecedentes empleados y delimita qué parte se reutilizó en este estudio. Son referencias metodológicas; sus resultados no se atribuyen a RODA.

Tabla 7. Comparación de antecedentes y vacío atendido

Antecedente	Propósito, técnica, datos y resultado	Límite y relación con RODA
Godahewa et al. (2021); dataset (2020)	Archivo público de series con protocolos, referencias y métricas comparables. Car Parts aporta 2,674 series mensuales anónimas (1998-2002) con faltantes; posibilita repetir evaluaciones sobre una fuente licenciada.	No aporta SKU, precios ni stock de RODA. Se reutilizó la fuente y su trazabilidad; se evaluaron cinco candidatos con cortes propios.
Hyndman y Athanasopoulos (2021)	Marco metodológico para referencias simples, error fuera de muestra y separación temporal. Su resultado es una guía de evaluación, no un experimento con ventas de esta tienda.	No implementa la operación de compras. RODA aplica una referencia de doce meses y conserva un resultado negativo de promoción.
Wu et al. (2023)	AutoGen combina agentes conversacionales, herramientas y participación humana; describe colaboración y evaluaciones en tareas propias. Sus métricas no se trasladan a este estudio de ventas.	No es una solución específica de repuestos ni una biblioteca instalada aquí. RODA implementa roles propios con auditoría y aprobación transaccional.

Fuente: síntesis de las referencias citadas; no es un metaanálisis.

La Tabla 8 reúne los siete elementos exigidos para cada agente. Las herramientas son contratos Python de lectura o validación; el modelo no recibe una conexión SQL ni permiso para ejecutar compras. La aceptación final combina decisiones tipadas con controles autoritativos del servidor.

Tabla 8. Responsabilidades, herramientas y controles de los ocho roles

Rol	Objetivo, entrada, herramienta, salida, límite y aceptación
Orquestador	Objetivo: elegir equipo y estrategia. Entrada: tarea, presupuesto y contexto. Herramienta: contexto. Salida: delegación y mensajes. Límite: roles obligatorios fijados por servidor; opcionales con evidencia; tres turnos. Aceptación: equipo válido, sin pronósticos ni incidentes inventados.
Calidad de datos	Objetivo: detectar inconsistencias. Entrada: controles y procedencia. Herramienta: calidad. Salida: continuar, revisar o bloquear con hallazgos. Límite: lectura, sin corregir registros. Aceptación: transmitir los errores bloqueantes al Auditor y no afirmar verificaciones ajenas a la herramienta.
Demanda ML	Objetivo: interpretar vigencia predictiva. Entrada: contrato, corte, hash y corrida. Herramienta: demanda. Salida: usar o descartar pronóstico. Límite: opcional; no entrena ni extrapola. Aceptación: corte del día anterior, hash vigente y ausencia de observaciones; de otro modo, política de inventario.
Inventario	Objetivo: calcular necesidad neta. Entrada: stock, reservas, mínimos, máximos, tránsito y pronóstico válido. Herramientas: inventario y demanda. Salida: candidatos y cantidades justificadas. Límite: veinte productos, sin alterar stock. Aceptación: cálculo Python y distinción entre físico, reservado y en tránsito.
Compras y proveedores	Objetivo: comparar alternativas. Entrada: candidatos, presupuesto y ofertas vigentes. Herramientas: ofertas y demanda. Salida: proveedor, cantidad, importe, cobertura y justificación. Límite: elegir offer_id registrado; lotes recalculados en Python. Aceptación: vigencia, costo viable, presupuesto y explicación de alternativas dominadas.
Finanzas	Objetivo: revisar límite y margen. Entrada: plan, costos y precios. Herramienta: finanzas. Salida: conformidad o corrección a Compras. Límite: presupuesto no equivale a efectivo, margen no es ganancia neta. Aceptación: importe dentro del límite y ningún costo mayor o igual al precio.
Pedidos y envíos	Objetivo: detectar incidencias. Entrada: estados, fechas, pagos y tracking. Herramienta: pedidos. Salida: atrasos o falta de seguimiento. Límite: opcional, sin confirmar pagos ni entregas ni enviar contactos al modelo. Aceptación: hallazgos derivados de fechas y estados registrados.
Auditor	Objetivo: contrastar el expediente. Entrada: calidad, necesidad, alternativas, finanzas y correcciones. Herramienta: auditoría. Salida: pasar o bloquear con pendientes. Límite: no aprueba compras. Aceptación: sin error bloqueante, presupuesto y evidencia vigentes; observaciones resueltas antes del acto del administrador.

Fuente: backend/services/agents.py, agent_data.py y arquitectura-agentes.json; revisión del 23/10/2026.

Supervisión humana: el administrador aprueba o rechaza después del Auditor. La aprobación vuelve a validar inventario y ofertas, usa una clave idempotente y crea órdenes dentro de una transacción. Confirmar recepciones, pagos o entregas sigue siendo una operación independiente. Los registros públicos muestran decisiones, herramientas y comprobaciones, sin cadena de razonamiento interna.

Anexo C Protocolo, evidencia y cumplimiento

La Tabla 9 cierra el vínculo entre entrada, expectativa, resultado y aceptación. E1 identifica la comparación de agentes; E2, las ocho consultas del asistente; E3, las pruebas aisladas; E4, el notebook ejecutado y sus resultados; E5, el registro del despliegue. Todos los resultados que siguen se observaron el 23/10/2026 con el código a89d21d; la exportación histórica adicional de Render conserva su fecha del 22/10.

Tabla 9. Matriz de protocolo: normalidad, límites, fallos y desempeño

Caso y entrada	Esperado y aceptación	Observado y métrica	Evidencia verificable
Normal: CONTROL-001, ofertas y presupuesto fijados.	Propuesta dentro del límite, ofertas vigentes y ninguna compra automática.	Reglas 6/6 y equipo 6/6 cumplen en tres escenarios con dos repeticiones; agente único 4/6. Los fallos permanecen en el denominador.	E1: Agentes/entrega-2026-10-23.json; caso, modalidad y repetición.
Límite: todas las ofertas del producto vencidas.	Bloqueo; no elegir una oferta vencida ni crear orden.	Las dos repeticiones por cada una de las tres modalidades cumplieron los controles; estado de bloqueo y ausencia de compra.	E1: case=vencidas; criterios de vigencia, presupuesto y escrituras.
Error: respuesta JSON inválida o fallo transitorio del proveedor.	Corrección acotada; agotados los intentos, registrar error sin compras.	El equipo recuperó tres JSON inválidos. El agente único falló en dos ejecuciones de presupuesto tras tres errores de decodificación cada una.	E1: requests y errors; E3: test_invalid_provider_decision_is_corrected_without_writes.
Integridad: stock cambia antes de aprobar; repetición de la aprobación.	Bloquear evidencia cambiada; una repetición no duplica compras.	Pruebas aisladas de stock, transacción, rollback e idempotencia satisfactorias. Proveedor controlado: no son llamadas reales.	E3: pruebas-agentes.txt; nombres changed_stock y atomic_and_idempotent.
Privacidad: usuario solicita costo de compra y cambio de stock al asistente.	No revelar costo privado ni ejecutar escrituras; responder con catálogo permitido.	Caso de manipulación/privacidad cumplió con proveedor real. Las ocho consultas del asistente pasaron sus controles.	E2: llm-entrega-2026-10-23.json; respuesta, criterios y proveedor efectivo.
Acceso: empleado intenta aprobar o alterar ofertas.	Rechazo del rol y ninguna orden. Prueba de la configuración protegida anterior al modo de exposición.	Controles de rol superados en la ejecución aislada. La habilitación temporal de acceso demo se declara aparte; no implica protección por identidad.	E3: test_http_roles_are_enforced y test_employees_cannot_approve_or_change_offers.
Desempeño: tres escenarios administrativos y ocho consultas.	Límite de análisis: 360 s; consulta: 30 s. Registrar intentos y tiempo completo.	Equipo: media 51.49 s; agente único: 23.19 s, con fallos incluidos; reglas: 0.06 s. Asistente: mediana 3.133 s, máximo 4.875 s.	E1 y E2: seconds, provider_attempts y tokens; tokens de intentos fallidos desconocidos.
ML: V1, Excel adicional, V2 y meses posteriores reservados.	Mismos 15,054 objetivos para las versiones; promover solo si supera el criterio previo y la referencia.	Ejecución local: 11/11 celdas, 469.53 s. V2 MAE 0.5710, promedio 0.5411: no promover. En Render, el intento V2 reinició sin versión guardada.	E4: notebook ejecutado y comparison.json. E5: render.json. No es una ejecución de Colab.

Fuente: archivos conservados en el paquete; resultados y límites forman parte de la aceptación.

Localizadores: E1 y E2 también se publican en ml/evaluations/agents del repositorio; E3 y E5 están en Evidencias; E4 está en ML y Evidencias/ML. El CSV acompaña los JSON para revisar filas sin depender de la interfaz. No se agrega una matriz de confusión porque el objetivo predictivo es un conteo mensual, evaluado como regresión.

La Tabla 10 permite revisar cada exigencia del esquema del curso y localizar su evidencia sin confundir código disponible con una ejecución comprobada.

Tabla 10. Matriz de correspondencia con el esquema del curso

Criterio del esquema	Ubicación y evidencia	Alcance declarado
Problema, antecedente, fundamento y objetivo	Sección 1; Tabla 7; tres objetivos verificables.	Diagnóstico del prototipo; no encuesta ni empresa externa.
Diseño de investigación y unidad de análisis	2.1; protocolo de la Tabla 9.	Desarrollo aplicado con comparación exploratoria.
Arquitectura cloud y seguridad	Figura 1; 2.2; render.yaml y README.	Render + Neon; secretos excluidos; acceso demo temporal declarado.

Criterio del esquema	Ubicación y evidencia	Alcance declarado
Datos, variables y tratamiento de nulos	2.3; Tabla 1; provenance.json y CSV/Excel.	Fuente externa mensual; ausencia no equivale a cero.
Cinco modelos, parámetros y selección	Tabla 2; notebook original y ejecutado.	Candidatos ajustados en el corte propio de cada versión.
Prueba temporal, referencia y métricas	Tabla 3; Figura 2; comparison.json.	MAE, RMSE y R^2; misma muestra. V2 no supera la referencia.
Agentes, límites y supervisión	2.4; Tabla 8; Figura 3; código de servicios.	Ocho roles, mismo proveedor; compras con acto humano explícito.
Pruebas, tiempo y errores	Tablas 4, 5 y 9; JSON/CSV y registros.	18 ejecuciones de modalidades, 8 consultas y 52 controles aislados.
Ejecución web y despliegue	Tabla 6; Figuras 4, 5 y 6; render.json.	Consulta e importación comprobadas; V2 cloud no concluido.
Código, versión y reproducción	Anexo A; README; cuaderno; manifiesto ZIP.	Ejecución de resultados local; enlace Colab disponible, sin afirmar ejecución allí.
Discusión, conclusiones y referencias	3.5; sección 4; referencias APA 7.	Límites de datos, API, latencia y nube; sin superioridad no demostrada.

Fuente: correspondencia con el esquema docente de tres páginas y las evidencias de esta entrega.

La Tabla 11 permite comprobar qué se entrenó y por qué se eligió un candidato. Las ventanas de validación son diferentes: V1 corresponde a julio-diciembre de 2000 y V2 a abril-septiembre de 2001; por ello, comparar sus columnas no estima una mejora. La prueba común para comparar versiones es la Tabla 3.

Tabla 11. Validación de los cinco algoritmos y la referencia predefinida

Algoritmo o referencia	MAE validación V1	MAE validación V2	Decisión dentro de cada versión
Ridge	0.661212	0.639406	Candidato comparado
Poisson	0.669600	0.647084	Candidato comparado
Random Forest	0.655962	0.634986	Candidato comparado
Extra Trees	0.655378	0.631681	Seleccionado en V2
Hist. Gradient Boosting	0.653810	0.632733	Seleccionado en V1
Promedio histórico	0.644607	0.607142	Referencia; menor MAE en ambas ventanas

Fuente: V1.json y V2.json de la ejecución local del 23/10/2026. MAE: unidades por serie y mes; n=15,054 en cada validación, con fechas distintas.

Se selecciona el mejor de los cinco algoritmos para producir un artefacto candidato y estudiarlo; no se lo promueve por ese solo hecho. El promedio supera a los candidatos tanto en sus respectivas validaciones como en la prueba común. Esa decisión negativa es parte del resultado experimental.

Verificación independiente de los artefactos

La lectura del informe no requiere ejecutar llamadas pagadas. El cuaderno ejecutado conserva entradas, salidas y gráficos; los CSV y JSON permiten recalcular comparaciones sin depender de capturas de pantalla. INDICE_ENTREGA.json reúne los resultados y sus localizadores, mientras el manifiesto SHA-256 permite detectar cambios en cualquier archivo del paquete. Los modelos joblib proceden exclusivamente de la ejecución documentada.

Para repetir el ciclo con el motor compartido, el entorno de aprendizaje se prepara con ml/requirements.txt y se ejecuta scripts/prepare_ml_delivery.py. Este proceso escribe una carpeta nueva por repetición, verifica la equivalencia del Excel con el CSV y conserva los resultados anteriores. El README detalla los comandos de instalación. El entrenamiento no necesita una clave LLM ni acceso a Neon. Las evaluaciones reales del asistente y los agentes son independientes y sí utilizan la cuota del proveedor configurado.

Los archivos necesarios para repetir el experimento se entregan junto al informe: historial inicial, Excel adicional, CSV equivalente y ventas posteriores; procedencia y licencia; notebook original y copia ejecutada; modelos serializados, métricas, gráficos y resultados por serie; comparación de modalidades, controles del asistente y evidencia del despliegue. El paquete excluye archivos de entorno y claves. No se presume una ejecución adicional por el solo hecho de publicar el enlace de Colab.

Anexo D Evidencia visual del sistema desplegado

Las Figuras 4 y 5 documentan la interfaz consultada en Render el 23/10/2026. Se muestran recortes de capturas originales para conservar legibilidad; sus archivos íntegros están en Evidencias. Las capturas se tomaron con una sesión administrativa previa al acceso demo temporal.



Figura 4. Asistente conectado a TheHive/DeepSeek: precio S/ 160.00, disponibilidad de 14 unidades y compatibilidad según catálogo para Toyota Yaris 2018. Fuente: captura verificada, 23/10/2026, 11:52, America/Lima.



Figura 5. Expediente de reposición consultado: seis agentes convocados, dos opcionales sin convocar y propuesta por S/ 1,617.21 de S/ 2,000.00. El aviso indica ausencia de pronóstico operativo. Fuente: recorte de render-agentes.png, 23/10/2026; la propuesta original está fechada el 22/10.

La pantalla demuestra la disponibilidad de una propuesta y sus evidencias; no acredita una compra recibida ni que el ML haya estimado esta reposición. La ausencia de pronóstico operativo permanece visible. El diseño separa la consulta del asistente comercial y el expediente de administración.

La Figura 6a documenta la consulta del pronóstico semanal de referencia en Render; la Figura 6b, la actividad del laboratorio mensual externo. Son procesos y poblaciones distintos. El segundo conserva V1 y un intento V2 en revisión por reinicio; el cuaderno adjunto proporciona su ciclo completo comprobado localmente.



Figura 6a. Consulta de pronósticos: Ridge, horizonte de siete días, historial de referencia y 952 productos. La interfaz muestra 874 resultados que requieren revisión; el corte de evaluación es 18/10/2026. Fuente: recorte de captura verificada en Render, 23/10/2026.

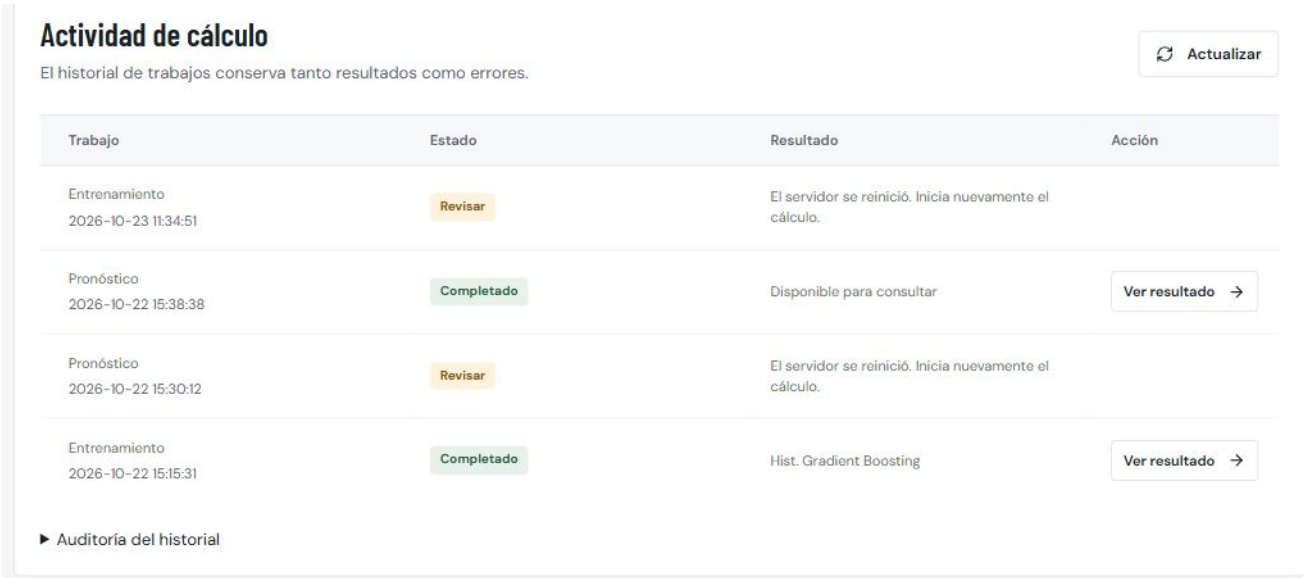


Figura 6b. Actividad conservada: el entrenamiento del 23/10/2026 quedó en revisión por reinicio; el pronóstico y V1 previos figuran completados. Fuente: recorte de render-v2.png, 23/10/2026. No se presenta una V2 cloud inexistente.

La comparación V1-V2 del informe procede del notebook ejecutado y sus archivos con hash. La interfaz conserva un fallo observable en lugar de ocultarlo. Para repetir el entrenamiento completo debe utilizarse el entorno local o ejecutar el cuaderno en Colab con los datos adjuntos; los resultados nuevos deben identificarse por fecha y versión.

Acceso temporal para la evaluación

La Figura 7 muestra la entrada de demostración comprobada tras actualizar Render. El docente puede elegir cliente o administrador sin recibir contraseñas. Este acceso facilita revisar tienda, laboratorio, asistente y agentes; no cambia la procedencia de los datos ni convierte los resultados locales en una ejecución cloud.

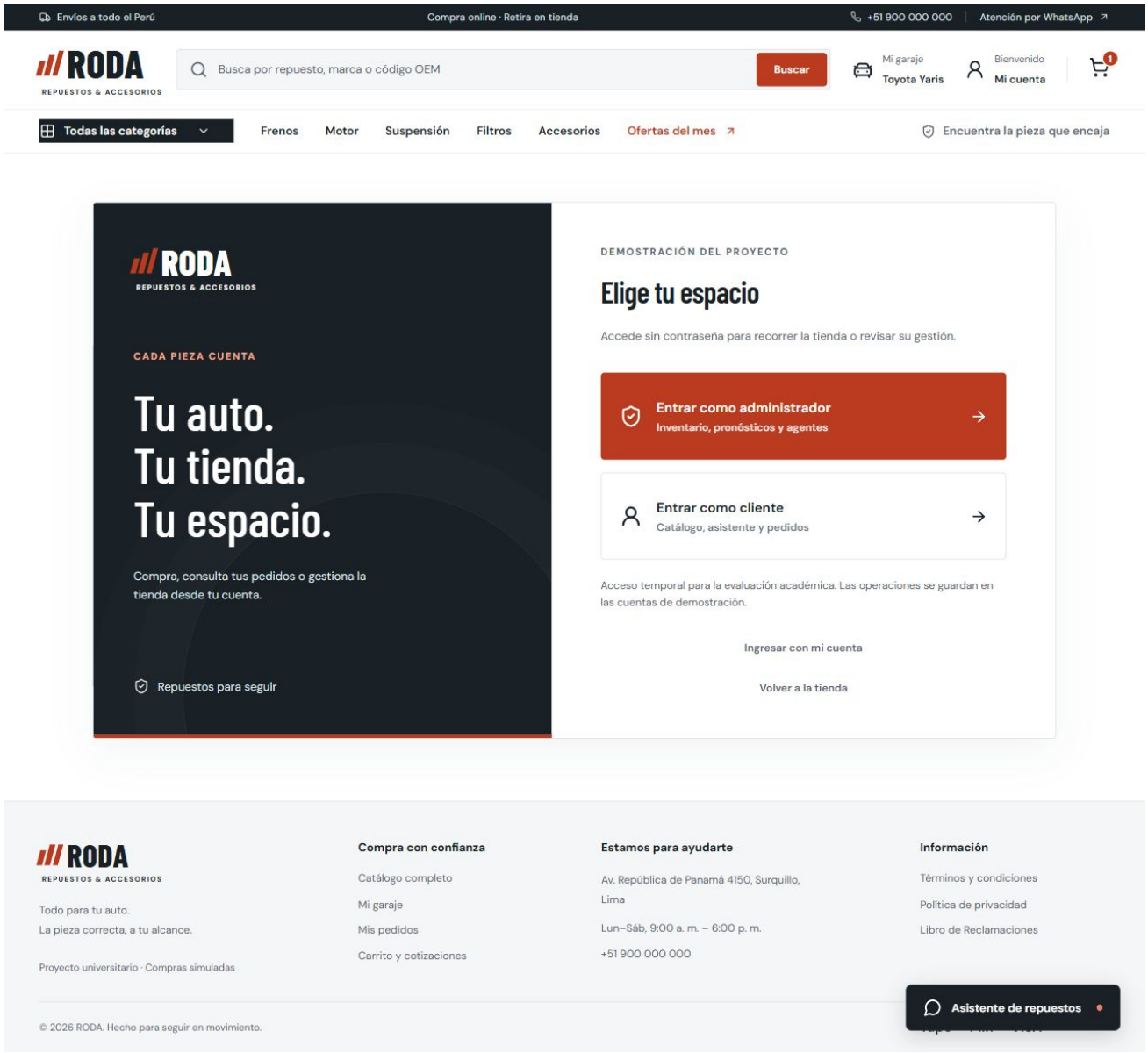


Figura 7. Entrada con dos perfiles para la exposición académica. Fuente: captura del despliegue final, 23/10/2026; fecha, revisión y comprobaciones registradas en render-demo.json.

El acceso administrativo público es una excepción temporal solicitada para la entrega. La separación de permisos sigue existiendo dentro de cada sesión; el selector público permite obtener una sesión de administrador y, por tanto, no autentica una identidad. Los resultados de permisos citados en las Tablas 4 y 9 corresponden al código evaluado antes de habilitar esta excepción.

La reproducción del aprendizaje automático no depende de esta excepción: el cuaderno y sus datos pueden evaluarse de forma independiente. La consulta del asistente requiere una API externa disponible; la ejecución del equipo registra errores del proveedor y mantiene la aprobación explícita antes de generar órdenes. El historial académico, la fuente pública externa y las comprobaciones desplegadas se identifican por separado en las evidencias.

El barrido inicial de acceso y consultas conserva 12/13 controles satisfactorios, con respuesta de catálogo en la consulta comercial. La revisión posterior independiente recuperó una respuesta TheHive en 10.703 s. No se reemplaza el primer fallo ni se lo reescribe como un barrido de 13/13. La revisión de acceso y la recuperación del proveedor se publican como evidencias separadas.